Dhaka International University

Department of Computer Science and Engineering

Lab Report

Course Name : Computer Graphics & Multimedia Lab
Course Code : CSE-406
Report Number : 02
Project Title : Implementation OpenGL and Create a 3D table and different camera view.

Submitted By:	Submitted To:
Name: MD.Rasel Islam Roll :25 Batch : D-72 Semester: 11th Dept. of CSE	Name: Umme Niraj Mahi Lecturer, Dept. of CSE Dhaka International University

Submission Date: 15 November 2025

Objective

1. Create a 3D table model using cubes in OpenGL.
2. Implement camera controls to view the table from multiple angles.
3. Understand transformations, model-view-projection matrices, and shader-based rendering.

Materials and Software

1. Desktop/Laptop with GPU supporting OpenGL 3.3+
2. Python 3.13
3. PyOpenGL library
4. glfw library
5. numpy library
6. IDE/Editor: Visual Studio Code

Introduction:

This lab focuses on creating a 3D table using cubes in OpenGL and exploring it from multiple camera angles. It demonstrates how model-view-projection matrices, shaders, and camera transformations affect 3D visualization. The experiment highlights real-time interaction and depth perception in a 3D environment.

Description:

- OpenGL Rendering Pipeline: Vertices are defined in object space and transformed using Model, View, and Projection matrices.
- Camera: A virtual camera is implemented to allow navigation around the scene. Camera transformations are handled with a look-at matrix.
- Shaders: Vertex and fragment shaders handle vertex transformation and color assignment.
- Table Construction: A table is constructed using scaled cubes—one for the tabletop and four for the legs.

Work Process:

1. Setup Window:
    - ➢ Initialized GLFW window with size 1280x720.
    - ➢ Enabled depth testing using glEnable(GL_DEPTH_TEST) to handle proper occlusion of cubes.
2. Shader Compilation:
    - ➢ Created vertex and fragment shaders for coloring and 3D transformations.
    - ➢ Linked shaders into a program.
3. Cube Data:
    - ➢ Defined vertices for a unit cube including color attributes.
4. Table Model:
    - ➢ Table consists of one tabletop and four legs, each represented by a scaled cube.
5. Table dimensions:
    - ➢ Top: 3 x 0.2 x 2 units
    - ➢ Legs: 0.2 x 1.0 x 0.2 units
6. Camera Implementation:

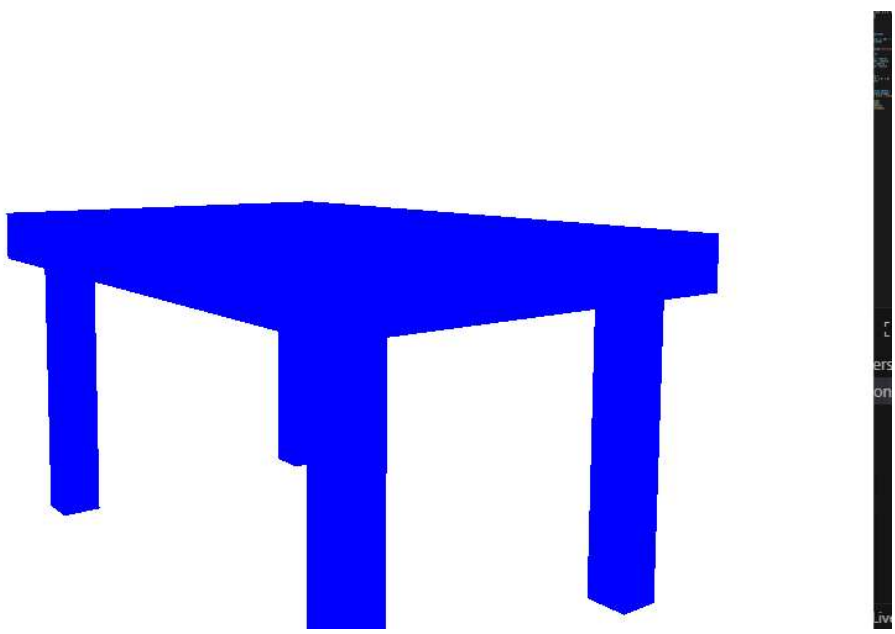
➢ Camera class with position, yaw, pitch, front, up, and right vectors.

7. Keyboard controls:
    - F → Front view
    - B → Back view
    - L → Left view
    - R → Right view
    - T → Top view
    - D → Bottom view
8. Rendering Loop:
    - ➢ Clear color and depth buffers.
    - ➢ Apply camera view and perspective matrices.
    - ➢ Render all cubes for the table.
    - ➢ Poll for keyboard input to change camera angle in real time.

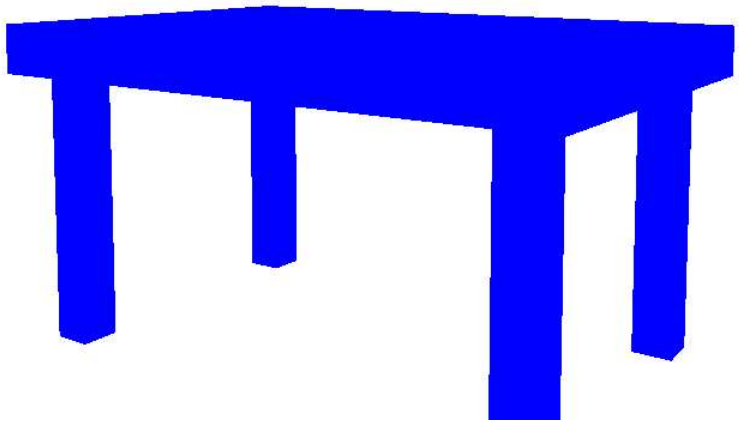
9. Observations Front View (F):
    - ➢ Table top visible with legs partially hidden behind the top.
    - ➢ Proper perspective makes the table appear 3D.
10. Back View (B):
    - ➢ Rear side of the table visible; demonstrates correct depth testing.
11. Left View (L) & Right View (R):
    - ➢ Side view highlights table thickness and leg positions.
12. Top View (T):
    - ➢ Clear view of tabletop surface and leg placement.
13. Bottom View (D):
    - ➢ Legs visible; tabletop appears thin.
    - ➢ Demonstrates that objects behind the camera are rendered correctly.

Screenshot:

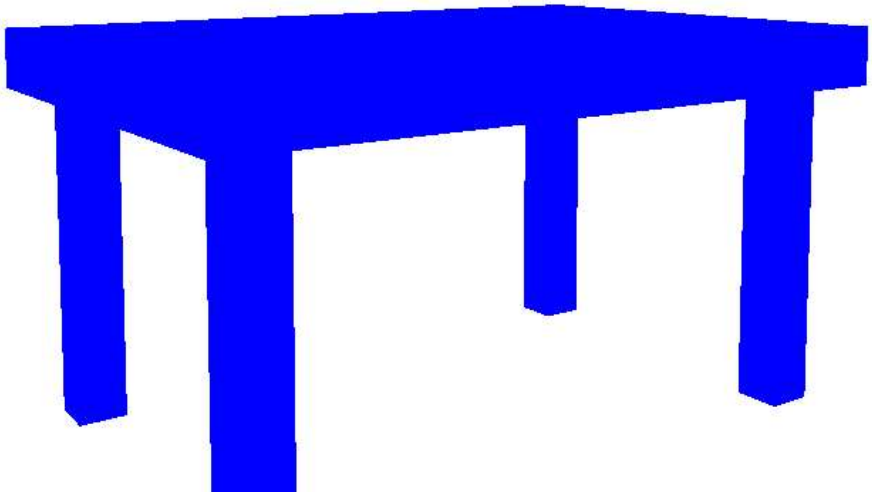
1. **Front view**



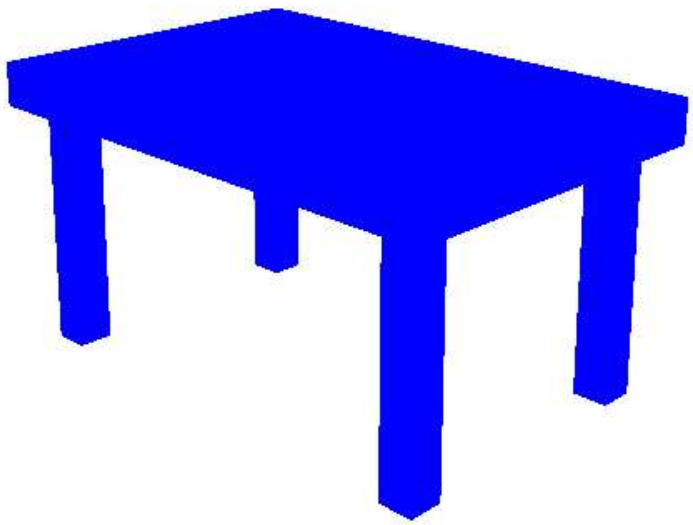
2. **Left side view**



3. **Right side view**



4. **Zoom out view**



Conclusion:

In this lab I successfully learned how to create a 3d table model and implement different camera angle views . This camera angle view like look at , front view, left, right side view , top, down view etc . This lab implement i am face problem for camera angle command programm not perfect , then i am need help google and AI(chatgpt, gemini) and successfully run and implement different view angel command .

This lab was very useful because it gave us a hands-on introduction to computer graphics.Though the shapes we created were simple camera aviews , they represent the foundation of more advanced graphics such as 3D modeling, animations, and interactive simulations. By understanding OpenGL at a basic level, we are now prepared to move forward into more complex topics like 3D rendering , texture mapping , lighting and shaders.